Statistical Github Repository

Practical 1

To create and display a student-score data frame using Python and pandas.

CODE

```
import pandas as pd


data = {
    "Student": ["Aarav", "Priya", "Rohit", "Sneha", "Kunal", "Divya"],
    "Score": [78, 85, 92, 67, 74, 88]
}


df = pd.DataFrame(data)


print(df)
```

Practical 2

To calculate the measures of central tendency mean, median and mode for the given sales data

CODE

```
import pandas as pd


sales = pd.Series([42, 38, 51, 29, 46, 34, 61, 40, 37, 44])


print("Mean:", sales.mean())
print("Median:", sales.median())
print("Mode:", sales.mode().tolist())
```

Practical 3

To calculate the sample variance and sample standard deviation for the given sales data.

CODE

```
import pandas as pd


sales = pd.Series([42, 38, 51, 29, 46, 34, 61, 40, 37, 44])
```

```
variance = sales.var()

std_dev = sales.std()


print(f"Sample Variance: {variance:.2f}")

print(f"Sample Std Dev: {std_dev:.2f}")
```

Practical 4

To generate the Binomial PMF for n = 5 trials and probability of success p = 0.3, and visualize it using a bar chart.

CODE

```
import numpy as np

import scipy.stats as stats

import matplotlib.pyplot as plt


n, p = 5, 0.3


k = np.arange(0, n + 1)

pmf = stats.binom.pmf(k, n, p)


for val, prob in zip(k, pmf):

   print(f"P(X={val}) = {prob:.5f}")


plt.bar(k, pmf, color="steelblue", edgecolor="black")


plt.title("Binomial PMF (n=5, p=0.3)")

plt.xlabel("Number of Successes (k)")

plt.ylabel("Probability")


plt.grid(axis='y', linestyle='--', alpha=0.7)


plt.show()
```

Practical 5

To generate the Poisson PMF for an average rate of $\lambda = 4$ complaints per day and plot the probability distribution.

CODE

```
import numpy as np

import scipy.stats as stats

import matplotlib.pyplot as plt


lam = 4


k = np.arange(0, 15)


pmf_poisson = stats.poisson.pmf(k, lam)


print(f"P(X=3) = {stats.poisson.pmf(3, lam):.5f}")


plt.bar(k, pmf_poisson, color="orange", edgecolor="black")


plt.title(r"Poisson PMF ($\lambda=4$ complaints/day)")

plt.xlabel("Number of Events (k)")

plt.ylabel("Probability")


plt.grid(axis='y', linestyle='--', alpha=0.7)


plt.show()
```

Practical 6

To demonstrate how the distribution of sample means becomes more concentrated and approximately normal as sample size increases, using an exponential (non-normal) population.

CODE

```
import numpy as np

import matplotlib.pyplot as plt
```

```
np.random.seed(42)

# Generating non-normal skewed population (Exponential)
population = np.random.exponential(scale=500, size=100000)

sample_sizes = [10, 30, 100]

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for i, n in enumerate(sample_sizes):

  sample_means = [
    np.random.choice(population, size=n, replace=False).mean()
    for _ in range(1000)
  ]

  axes[i].hist(
    sample_means,
    bins=30,
    color="teal",
    edgecolor="black",
    alpha=0.7
  )

  axes[i].set_title(
    f"Sample Size n = {n}\nMean={np.mean(sample_means):.1f},
SD={np.std(sample_means):.1f}"
  )

  axes[i].set_xlabel("Sample Mean")
```

```
plt.tight_layout()

plt.show()
```

Practical 7

To compute 90%, 95% and 99% confidence intervals for the mean weight using the tdistribution

CODE

```
import pandas as pd


sales = pd.Series([42, 38, 51, 29, 46, 34, 61, 40, 37, 44])


print("Mean:", sales.mean())

print("Median:", sales.median())

print("Mode:", sales.mode().tolist())
```

Practical 8

To test whether the mean bottle volume differs significantly from the hypothesized population mean of 500 ml.

CODE

```
import numpy as np

from scipy import stats


bottles = np.array([498, 502, 497, 501, 499, 503, 496, 500])


t_stat, p_val = stats.ttest_1samp(bottles, popmean=500)


print(f"t-statistic: {t_stat:.3f}")

print(f"p-value: {p_val:.4f}")
```

Practical 9

To determine whether the mean performance of Shift A differs significantly from Shift B.

CODE

```
import numpy as np

from scipy import stats


shift_a = np.array([45, 48, 42, 50])
```

```
shift_b = np.array([52, 55, 50, 58])


t_stat, p_val = stats.ttest_ind(shift_a, shift_b)


print(f"t-statistic: {t_stat:.3f}")

print(f"p-value: {p_val:.4f}")
```

Practical 10

To compute the regression intercept, slope and R-squared for Sales predicted by TV Spend, using ordinary least squares.

CODE

```
import numpy as np

import pandas as pd

import statsmodels.api as sm


df = pd.DataFrame({

   "TV_Spend": [100, 150, 200, 250, 300],

   "Sales": [150, 200, 230, 280, 320]

})


# Add constant (intercept) column for OLS

X = sm.add_constant(df["TV_Spend"])


# Define dependent variable

y = df["Sales"]


# Fit OLS regression model

model = sm.OLS(y, X).fit()


# Display results

print(f"Intercept (beta_0): {model.params['const']:.2f}")

print(f"Slope (beta_1): {model.params['TV_Spend']:.2f}")

print(f"R-squared: {model.rsquared:.4f}")
```